

MINDANAO STATE UNIVERSITY – ILIGAN INSTITUTE OF TECHNOLOGY

BCA182 – Embedded Systems Programming Laboratory

LABORATORY ACTIVITY 1

Real-Time Multisensor Room Monitoring System

Laboratory Report

Prepared by:
Charles Daniel Estrada

Platform: STM32F103C8T6 Blue Pill • PlatformIO • STM32Cube HAL • FreeRTOS
Date: September 2026

3. FreeRTOS Architecture

The application uses multiple FreeRTOS tasks with explicit priorities and blocking delays. SensorTask uses `vTaskDelayUntil()` to maintain a two-second sampling period. MotionTask polls the PIR every 100 ms, while StateTask waits for a task notification and applies the inactivity rule.

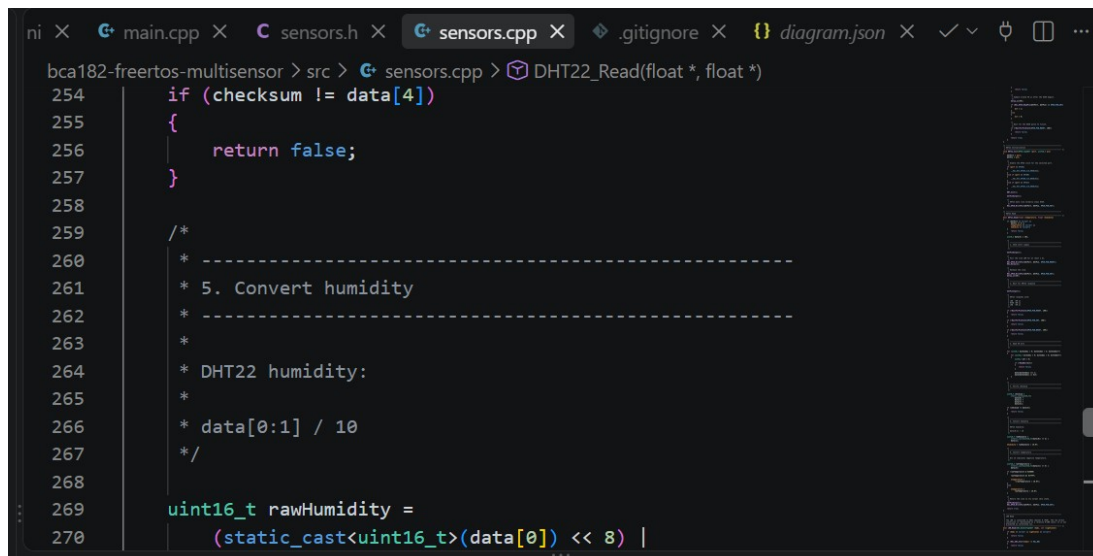
Task	Priority	Stack	Main responsibility
SensorTask	2	256 words	DHT22/LDR acquisition and sensor queue
MotionTask	3	256 words	PIR polling and motion notification
StateTask	2	256 words	ACTIVE/INACTIVE state machine
InputTask	3	256 words	Rotary encoder navigation
DisplayTask	2	352 words	OLED rendering from latest sensor data
AlarmTask	2	256 words	Temperature threshold and buzzer

Inter-task communication includes a four-element sensor queue and a task notification from MotionTask to StateTask. Mutex protection is used for shared display-mode state. Critical sections are used where a short atomic snapshot of shared motion information is required.

4. Implementation

4.1 Milestones 1-2: Sensors and Activity State

Milestone 1 implemented the DHT22 and LDR subsystem. The DHT22 driver performs the required GPIO timing and validates received data using the checksum. The LDR uses ADC1 channel 0 and converts the raw reading to a relative 0–100 light level rather than calibrated lux. SensorData contains temperature, humidity, light level, and motion status, and SensorTask places valid readings into a four-element queue.



```

ni X main.cpp X sensors.h X sensors.cpp X .gitignore X diagram.json X ✓ ▾ 🔍 📄 ...
bca182-freertos-multisensor > src > sensors.cpp > DHT22_Read(float *, float *)
254     if (checksum != data[4])
255     {
256         return false;
257     }
258
259     /*
260     * -----
261     * 5. Convert humidity
262     * -----
263     *
264     * DHT22 humidity:
265     *
266     * data[0:1] / 10
267     */
268
269     uint16_t rawHumidity =
270         (static_cast<uint16_t>(data[0]) << 8) |
  
```

Figure 2. DHT22 driver implementation shown in sensors.cpp.

Milestone 2 added the PIR and activity state machine. MotionTask runs at priority 3 with a 100 ms polling interval. StateTask starts ACTIVE, changes to INACTIVE after 15 seconds without motion, and returns to ACTIVE when MotionTask sends a task notification.

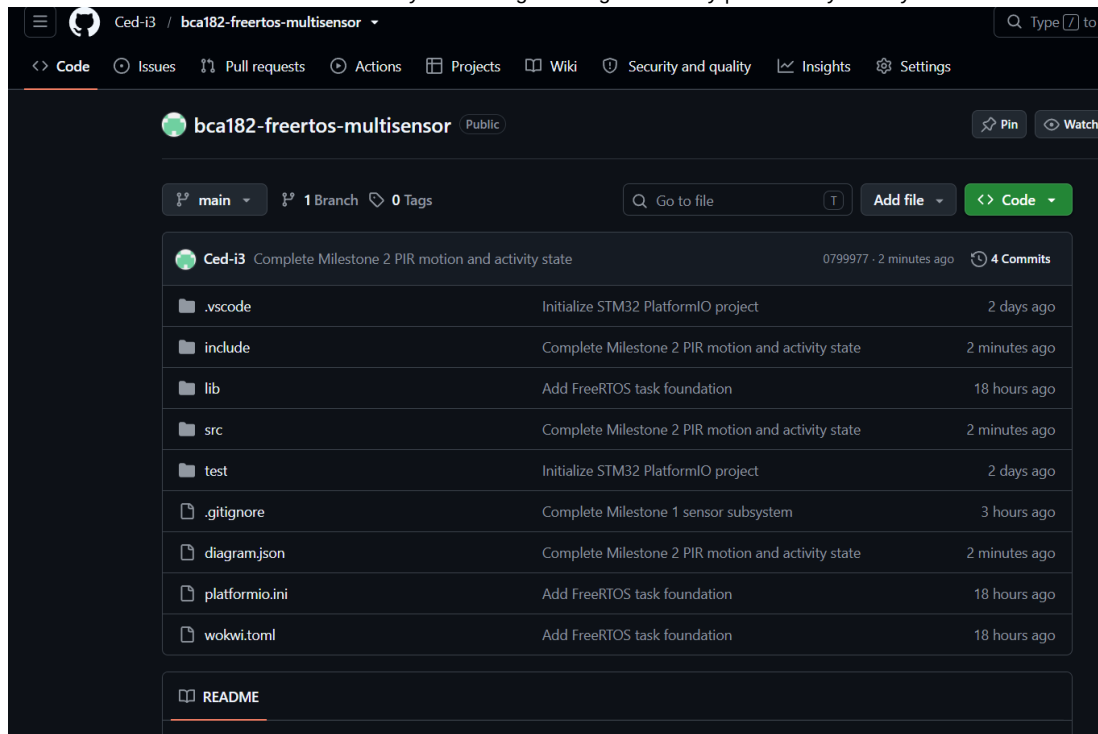


Figure 3. GitHub project history showing the Milestone 2 implementation.

4.2 Milestones 3–5: Input, Display, and Alarm

Milestone 3 added rotary-encoder navigation using PB10 and PB11. InputTask polls the encoder at 1 ms intervals with debouncing and protects the selected DisplayMode with a mutex. Navigation wraps around the four display modes and is ignored while the room is INACTIVE.

Milestone 4 added a bare-metal SSD1306 I2C driver using STM32 HAL, avoiding the Arduino framework. DisplayTask runs periodically, reads the latest sensor data without consuming the queue item, obtains the selected display mode and activity state, and renders the corresponding value.

Milestone 5 added AlarmTask and a buzzer on PB1. The alarm activates when temperature is below 18 °C or above 30 °C and is silenced while the system is INACTIVE.

4.3 Milestone 6: Unit Testing

The project includes a native Unity-based test environment for logic that can be tested without the STM32 hardware. Three test suites cover temperature evaluation, display-mode navigation, and system-state evaluation. The suite contains 13 meaningful test cases in total. A portable MinGW-w64 host compiler and small HAL/FreeRTOS mocks were used only for the native test build; the production STM32 architecture was not replaced by a native implementation.

Test suite	Cases	Coverage
Temperature evaluation	5	Normal, lower boundary, upper boundary, below range, above range
Display mode navigation	4	All four measurement modes and wraparound behavior
System state evaluation	4	ACTIVE/INACTIVE transitions and motion/inactivity conditions
Total	13	13 meaningful unit tests

5. Verification and Testing

Verification was performed through firmware compilation, native unit testing, static analysis, configuration inspection, and available Wokwi simulation behavior. The project also maintains incremental Git commits for the completed milestones.

Verification item	Result
STM32 PlatformIO build	Successful; no compiler errors or warnings
Native Unity tests	13/13 tests passed (5.523 s)
Static analysis (recorded)	0 high, 0 medium, 27 low findings; check passed
Git repository	main synchronized with origin/main; working tree clean
Wokwi serial output	Not used as primary verification because output was unreliable
OLED/other visual runtime output	Not treated as fully runtime-verified

The static-analysis low findings were primarily style-level C-style-cast notices and cross-source unused-function reports that did not correspond to a functional defect in the application. No high- or medium-level findings were recorded in the reviewed implementation.

File	Commit Message	Time Ago
.vscode	Initialize STM32 PlatformIO project	2 days ago
include	Complete Milestone 1 sensor subsystem	19 minutes ago
lib	Add FreeRTOS task foundation	15 hours ago
src	Complete Milestone 1 sensor subsystem	19 minutes ago
test	Initialize STM32 PlatformIO project	2 days ago
.gitignore	Complete Milestone 1 sensor subsystem	19 minutes ago
diagram.json	Complete Milestone 1 sensor subsystem	19 minutes ago
platformio.ini	Add FreeRTOS task foundation	15 hours ago
wokwi.toml	Add FreeRTOS task foundation	15 hours ago
README		

Figure 4. GitHub milestone history used as evidence of incremental version control.

5.1 Final Unit-Test Result

The final native test run completed successfully with all 13 test cases passing: test_display_mode, test_system_state, and test_temperature all reported PASSED. The complete run finished in approximately 5.5 seconds.

Environment	Test	Status	Duration
native	test_display_mode	PASSED	00:00:03.491
native	test_system_state	PASSED	00:00:02.501
native	test_temperature	PASSED	00:00:02.290
===== 13 test cases: 13 succeeded in 00:00:08.281 s =====			

Figure 10. Final native Unity test result: 13 test cases succeeded.

5.2 Problems and Limitations

During development, the main difficulties were unreliable Wokwi serial output and limited visibility of OLED/other visual outputs, which prevented those features from being treated as fully runtime-verified. The native unit-test setup also required a separate host compiler and small HAL/FreeRTOS mocks. The LDR reports a relative 0–100

light level rather than calibrated lux. These limitations were documented instead of claiming verification that could not be reliably observed.

6. Static Code Analysis

PlatformIO static analysis was used to inspect the project for common coding and structural issues. The reviewed implementation produced no high- or medium-severity findings. The remaining low-level findings were mainly style-related and tool-generated cross-source reports. These were reviewed rather than ignored blindly.

7. Engineering Discussion

The project demonstrates the separation of responsibilities expected in a real-time embedded system. Sensor acquisition is isolated from presentation and alarm logic, while FreeRTOS provides scheduling and inter-task communication. `vTaskDelayUntil()` is appropriate for the periodic sensor workload because it maintains a regular sampling schedule without busy waiting.

The queue prevents sensor producers and consumers from depending on unprotected shared sensor variables. The task notification provides a lightweight event mechanism for motion activity. Mutex protection of display selection prevents concurrent access to the navigation state.

The principal verification limitation is the current Wokwi environment: serial output was unreliable and OLED/other visual outputs could not be treated as consistently observable. Therefore, the report does not claim full runtime verification for outputs that could not be reliably observed. The LDR value is also a relative simulated light level rather than calibrated lux.

Git history was maintained incrementally. The major milestone commits include Milestone 1 sensor subsystem, Milestone 2 PIR/activity state, Milestones 3–5 input/display/alarm, Milestone 6 native unit tests, and the final production-build cleanup commit 9d6f2ff.

8. Conclusion

The laboratory activity resulted in a FreeRTOS-based multisensor room monitoring system for the STM32F103C8T6 Blue Pill. The implementation integrates environmental sensing, motion detection, activity-state management, rotary navigation, OLED presentation, and temperature alarm logic using STM32 HAL and native FreeRTOS APIs.

The project also demonstrates periodic scheduling, queues, task notifications, mutex protection, state-machine logic, unit testing, static analysis, and incremental Git-based development. The final cleanup commit excludes the test-only HAL mock from the STM32 production build. The final cleanup commit excludes the test-only HAL mock from the STM32 production build. Runtime limitations in Wokwi are explicitly documented so that the report distinguishes implemented functionality from functionality that could not be reliably observed in simulation.

Appendix A – Selected Evidence from the Submitted Milestone Document

The following figures were retained from the submitted milestone document because they directly document the development process and implementation.

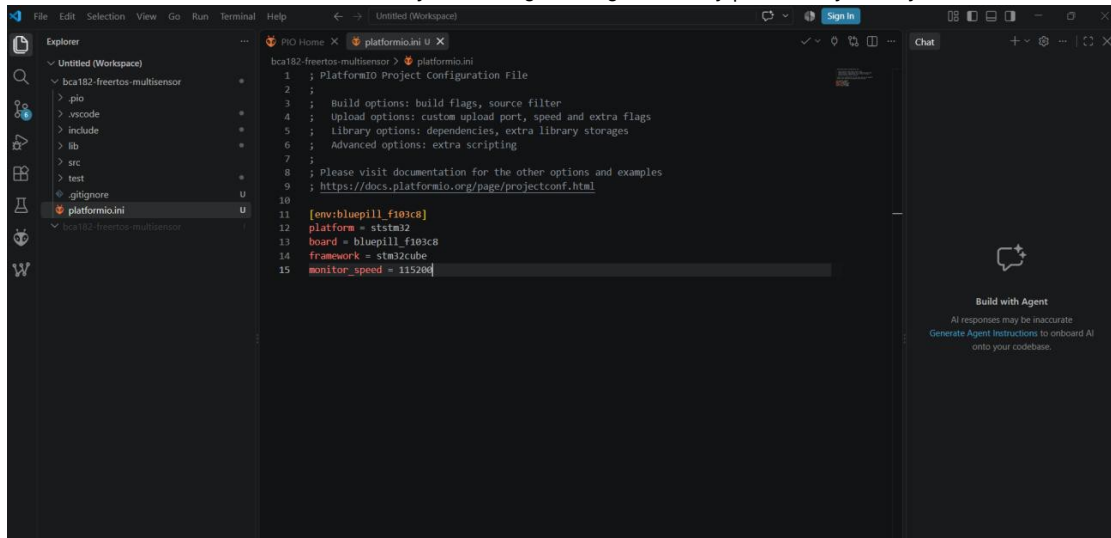


Figure 5. PlatformIO/VS Code project setup.

```

create mode 100644 lib/FreeRTOS/include/timers.h
create mode 100644 lib/FreeRTOS/portable/GCC/ARM_CM3/port.c
create mode 100644 lib/FreeRTOS/portable/GCC/ARM_CM3/portmacro.h
create mode 100644 lib/FreeRTOS/portable/MemMang/heap_4.c
create mode 100644 lib/FreeRTOS/src/croutine.c
create mode 100644 lib/FreeRTOS/src/event_groups.c
create mode 100644 lib/FreeRTOS/src/heap_4.c
create mode 100644 lib/FreeRTOS/src/list.c
create mode 100644 lib/FreeRTOS/src/port.c
create mode 100644 lib/FreeRTOS/src/queue.c
create mode 100644 lib/FreeRTOS/src/stream_buffer.c
create mode 100644 lib/FreeRTOS/src/tasks.c
create mode 100644 lib/FreeRTOS/src/timers.c
create mode 100644 wokwi.toml
PS C:\Users\NITRO V 16\Downloads\BCA182_LAB1Folder\bca182-freertos-multisensor> git push
Enumerating objects: 52, done.
Counting objects: 100% (52/52), done.
Delta compression using up to 16 threads
Compressing objects: 100% (43/43), done.
Writing objects: 100% (46/46), 214.01 KiB | 3.75 MiB/s, done.
Total 46 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/Ced-i3/bca182-freertos-multisensor.git
63ceff8..b1bba5b main -> main
PS C:\Users\NITRO V 16\Downloads\BCA182_LAB1Folder\bca182-freertos-multisensor> |

```

Figure 6. PlatformIO build/Git terminal evidence from the milestone record.

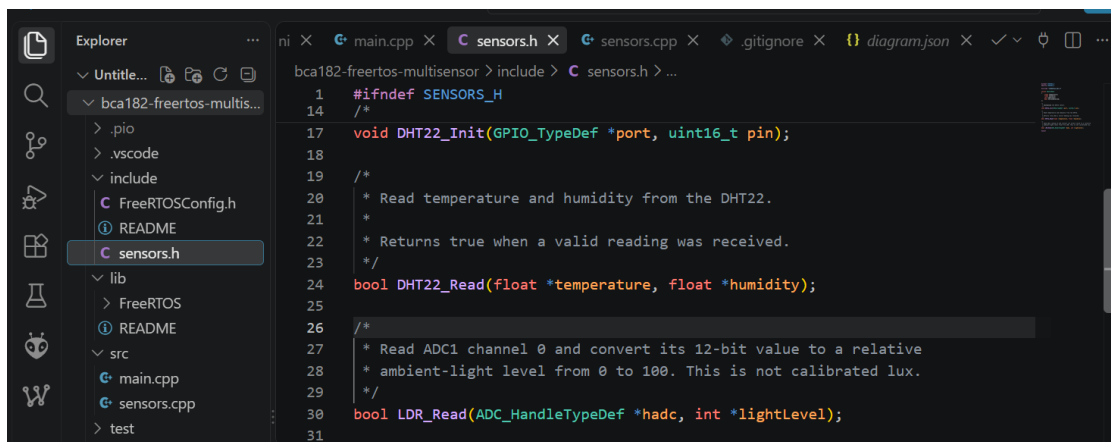


Figure 7. SensorData and sensor interface implementation.

Appendix B – Final Project Evidence

The following screenshots represent the final project state supplied for this report. They are included as final implementation evidence in addition to the milestone documentation.

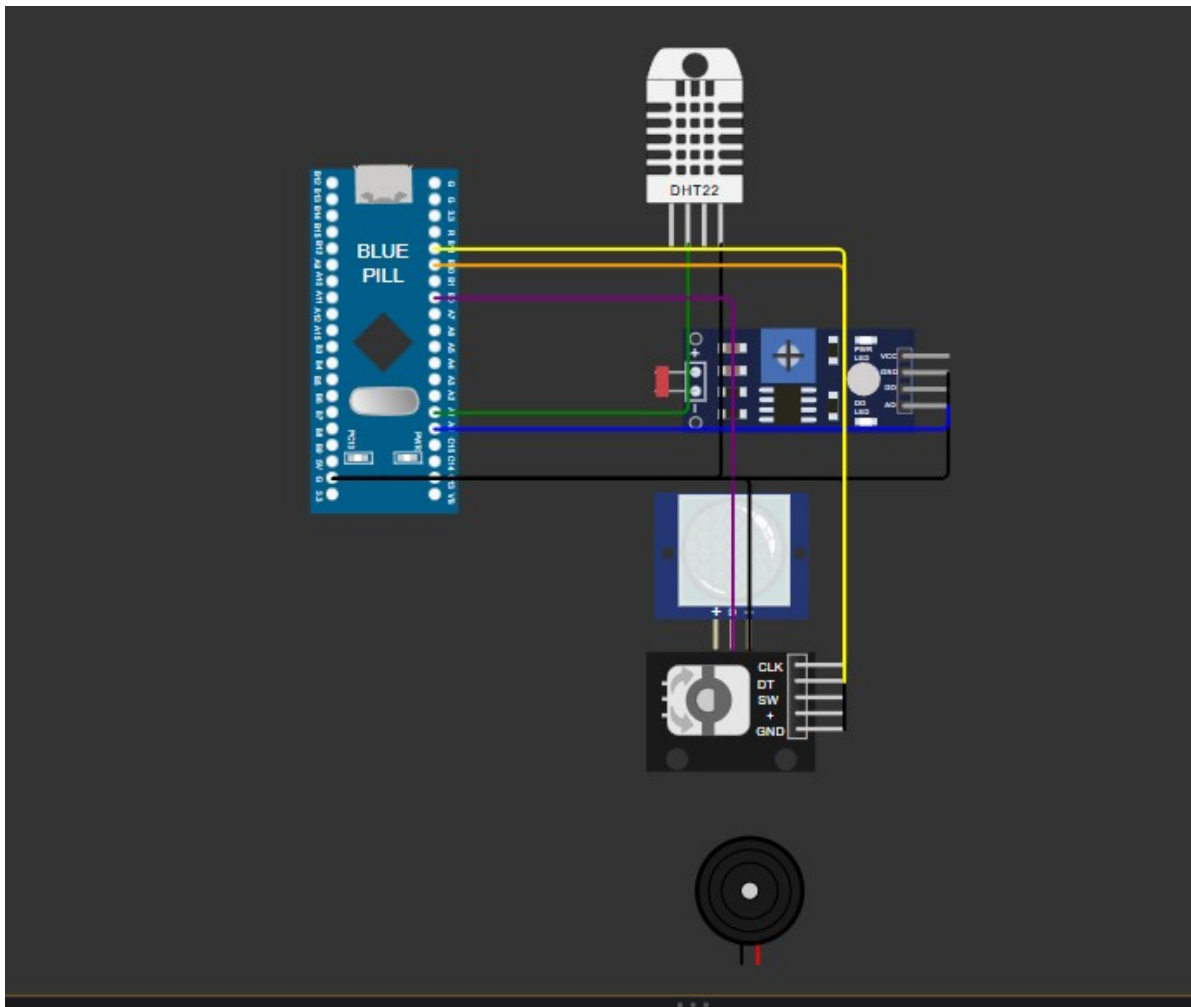


Figure 8. Final Wokwi circuit configuration showing the STM32 Blue Pill, DHT22, LDR, PIR sensor, rotary encoder, and buzzer.

The final circuit screenshot shows the integrated hardware layout used for the completed system. The components visible correspond to the project functions described in the implementation section.

Ced-i3 Add native unit tests for Milestone 6 209a7de · 19 minutes ago 7 Commits

File/Folder	Commit Message	Time
.vscode	Initialize STM32 PlatformIO project	4 days ago
include	Add unit-testable logic and 13 Unity tests	9 hours ago
lib	Add FreeRTOS task foundation	2 days ago
src	Add native unit tests for Milestone 6	19 minutes ago
test	Add native unit tests for Milestone 6	19 minutes ago
.gitignore	Complete Milestone 1 sensor subsystem	2 days ago
FreeRTOS.h	Add native unit tests for Milestone 6	19 minutes ago
build_compiler.py	Add native unit tests for Milestone 6	19 minutes ago
diagram.json	Complete Milestones 3-5 input display and alarm	14 hours ago
platformio.ini	Add native unit tests for Milestone 6	19 minutes ago
queue.h	Add native unit tests for Milestone 6	19 minutes ago
semphr.h	Add native unit tests for Milestone 6	19 minutes ago
stm32f1xx_hal.h	Add native unit tests for Milestone 6	19 minutes ago
task.h	Add native unit tests for Milestone 6	19 minutes ago
wokwi.toml	Add FreeRTOS task foundation	2 days ago

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Contributors 1
 Ced-i3

Languages
C 95.2% C++ 4.8%

Suggested workflows
Based on your tech stack
[MSBuild based projects](#) [Configure](#)
Build a MSBuild based project.
By GitHub Actions

Figure 9. Final GitHub repository state showing the completed Milestone 6 commit and project structure.

The final repository screenshot shows the project structure and the latest commits, including the final cleanup commit 9d6f2ff, providing evidence of the completed testing milestone and production-build cleanup.